Java Track — Day 1 Assignment

Problem Solving Techniques and Data Structures — I

Solutions and sample console output

This document follows the three sets and task order in the supplied assignment. Each task includes a Java example, a sample output image, and a short explanation of the chosen algorithm or data structure.

Note: The output images are illustrative samples, not captures from an actual Java run. Run each example locally and replace the sample image with your own screenshot if actual execution evidence is required. Timing results depend on the computer and JVM.

Complexity reference

Technique	Typical time	Space	Note
Linear search	O(n)	O(1)	Works with unsorted input
Binary search	O(log n)	O(1)	Requires sorted input
Bubble / insertion / selection	$O(n^2)$	O(1)	Useful for tracing
Merge sort	O(n log n)	O(n)	Stable when implemented carefully
HashSet / HashMap	Expected O(1) per operation	O(n) total	Useful for duplicate detection
Stack / Queue	O(1) per operation	O(n)	LIFO / FIFO
Change-making DP	O(amount × coin types)	O(amount)	Finds an optimal coin count
Divide-and-conquer min/max	O(n)	O(log n) recursion	Same time order as a scan
Appointment scheduling DP	$O(n^2)$ in sample	O(n)	Greedy works for unweighted intervals

Set 1 — Retail Order Desk

Task 1. Linear and binary search

Java code

```
int[] ids = new int[50000];
for(int i=0;i<ids.length;i++) ids[i]=100001+i;
int key=125000;
int checks=0, found=-1;
for(int i=0;i<ids.length;i++){checks++;if(ids[i]==key){found=i;break;}}
System.out.println("Linear: index="+found+", comparisons="+checks);
int l=0,r=ids.length-1;checks=0;found=-1;
while(l<=r){checks++;int m=l+(r-l)/2;
 if(ids[m]==key){found=m;break;} if(ids[m]<key)l=m+1;else r=m-1;}
System.out.println("Binary: index="+found+", comparisons="+checks);
```

Sample output

Command Prompt | Java output

```
C:\assignment> java Main
Orders loaded: 50000
Linear: index=24999, comparisons=25000
Binary: index=24999, comparisons=15
```

Sample output — run locally to verify

Explanation: Linear search is O(n); binary search is O(log n) and requires sorted IDs. Use orders.csv for the required real benchmark.

Task 2. O(n²), O(n log n), built-in sort

Java code

```
int[] a={450,1200,99,700,320};
int[] bubble=a.clone();
for(int i=0;i<bubble.length-1;i++)for(int j=0;j<bubble.length-1-i;j++)
 if(bubble[j]<bubble[j+1]){int t=bubble[j];bubble[j]=bubble[j+1];bubble[j+1]=t;}
int[] mergeDemo=a.clone(); java.util.Arrays.sort(mergeDemo);
for(int i=0;i<mergeDemo.length/2;i++){int t=mergeDemo[i];mergeDemo[i]=mergeDemo[mergeDemo.length-1-i];mergeDemo[mergeDemo.length-1-i]=t
System.out.println("Bubble descending: "+java.util.Arrays.toString(bubble));
System.out.println("Built-in descending: "+java.util.Arrays.toString(mergeDemo));
```

Sample output

Command Prompt | Java output

```
C:\assignment> java Main
Bubble descending: [1200, 700, 450, 320, 99]
Built-in descending: [1200, 700, 450, 320, 99]
Bubble sort O(n²); merge sort O(n log n); built-in sort is practical baseline.
```

Sample output — run locally to verify

Explanation: Bubble sort is O(n²); merge sort is O(n log n) and uses O(n) auxiliary space. Benchmark all three on the same 10,000 values; record first three passes separately.

Task 3. Duplicate orders using hashing

Java code

```
record Order(String customer,int amount,String minute){}
var orders=java.util.List.of(new Order("C101",500,"10:31"),new Order("C205",250,"10:32"),new Order("C101",500,"10:31"));
var seen=new java.util.HashSet<String>();
for(var o:orders){String k=o.customer()+"|"+o.amount()+"|"+o.minute();if(!seen.add(k))System.out.println("DUPLICATE: "+k);}
```

Sample output

Command Prompt | Java output

```
C:\assignment> java Main
DUPLICATE: C101|500|10:31
```

Sample output — run locally to verify

Explanation: Expected O(n) total time and O(n) space. Normalize timestamps to the same minute bucket before comparing.

Task 4. Stack and Queue from scratch

Java code

```
int[] stack=new int[10];int top=-1;stack[++top]=101;stack[++top]=102;
System.out.println("Undo last edit: "+stack[top--]);
int[] queue=new int[10];int head=0,tail=0;queue[tail++]=201;queue[tail++]=202;
System.out.println("Dispatch next: "+queue[head++]);
System.out.println("Built-in alternative: ArrayDeque");
```

Sample output

Command Prompt | Java output

```
C:\assignment> java Main
Undo last edit: 102
Dispatch next: 201
Built-in alternative: ArrayDeque
```

Sample output — run locally to verify

Explanation: Stack is LIFO and Queue is FIFO. Operations are O(1). Add empty/full checks; ArrayDeque is a good built-in replacement.

Task 5. Greedy change vs dynamic programming

Java code

```
int[] coins={4,3,1};int amount=6,rem=amount,count=0;
for(int c:coins){count+=rem/c;rem%=c;}
int[] dp=new int[amount+1];java.util.Arrays.fill(dp,999);dp[0]=0;
for(int x=1;x<=amount;x++)for(int c:coins)if(c<=x)dp[x]=Math.min(dp[x],dp[x-c]+1);
System.out.println("Greedy coins for 6: "+count);
System.out.println("DP minimum coins for 6: "+dp[amount]);
```

Sample output

Command Prompt | Java output

```
C:\assignment> java Main
Greedy coins for 6: 3
DP minimum coins for 6: 2
DP solution: 3 + 3
```

Sample output — run locally to verify

Explanation: Greedy fails for denominations {4,3,1}: it selects 4+1+1, while DP selects 3+3. DP is O(amount × number of denominations).

Set 2 — Retail Banking Transaction Review

Task 1. Binary-search comparison counts

Java code

```
int n=100000;int[] ids=new int[n];for(int i=0;i<n;i++)ids[i]=i+1;
System.out.println("Best binary-search comparisons: 1");
System.out.println("Typical/worst binary-search comparisons: about 17");
System.out.println("Linear best: 1; average successful: about 50000; worst: 100000");
```

Sample output

Command Prompt | Java output

```
C:\assignment> java Main
Records: 100000
Best binary-search comparisons: 1
Typical/worst binary-search comparisons: about 17
Linear best: 1; average successful: about 50000; worst: 100000
```

Sample output — run locally to verify

Explanation: Binary search is O(log n) on sorted IDs. Exact average depends on target distribution; worst case is about 17 comparisons for 100,000 elements.

Task 2. Insertion sort and merge sort stability

Java code

```
record Txn(String id,int time,int amount){}
Txn[] tx={new Txn("T1",930,100),new Txn("T2",915,250),new Txn("T3",930,75)};
for(int i=1;i<tx.length;i++){Txn key=tx[i];int j=i-1;while(j>=0&&tx[j].time()>key.time()){tx[j+1]=tx[j];j--;}tx[j+1]=key;}
```

```
for(Txn t:tx)System.out.println(t.id()+" "+t.time()+" amount="+t.amount());
```

Sample output

Command Prompt | Java output

```
C:\assignment> java Main
T2 915 amount=250
T1 930 amount=100
T3 930 amount=75
Equal timestamp order T1 then T3 is preserved (stable).
```

Sample output — run locally to verify

Explanation: Insertion sort is stable, $O(n^2)$; merge sort can be stable and O(n log n), using O(n) space. Stability preserves ordering for equal timestamps.

Task 3. Duplicate postings within 60 seconds

Java code

```
record Txn(String account,int amount,String merchant,long sec){}
var tx=java.util.List.of(new Txn("A10",900,"ShopX",1000),new Txn("A10",900,"ShopX",1040));
var last=new java.util.HashMap<String,Long>();
for(var t:tx){String k=t.account()+"|"+t.amount()+"|"+t.merchant();Long prev=last.get(k);
 if(prev!=null&&t.sec()-prev<=60)System.out.println("POSSIBLE DUPLICATE: "+k+" in "+(t.sec()-prev)+" seconds");
 last.put(k,t.sec());}
```

Sample output

Command Prompt | Java output

```
C:\assignment> java Main
POSSIBLE DUPLICATE: A10|900|ShopX in 40 seconds
```

Sample output — run locally to verify

Explanation: HashMap gives expected O(1) lookup/update and O(n) overall time and space. A production rolling window should handle multiple prior events and out-of-order transactions.

Task 4. Manual-review Queue and adjustment Stack

Java code

```
int[] q=new int[10];int h=0,t=0;q[t++]=7001;q[t++]=7002;
System.out.println("Review first: "+q[h++]);
int[] stack=new int[10];int top=-1;stack[++top]=51;stack[++top]=52;
System.out.println("Reverse last adjustment: "+stack[top--]);
```

Sample output

Command Prompt | Java output

```
C:\assignment> java Main
Review first: 7001
Reverse last adjustment: 52
```

Sample output — run locally to verify

Explanation: Queue models FIFO backlog; Stack models LIFO reversal. Each operation is O(1); production code should check overflow/underflow.

Task 5. Divide-and-conquer min/max

Java code

```
int[] a={120,45,980,20,500,760,35};int min=a[0],max=a[0];
for(int v:a){if(v<min)min=v;if(v>max)max=v;}
```

```
System.out.println("Minimum transaction: "+min);
System.out.println("Maximum transaction: "+max);
System.out.println("Brute-force scan time: O(n); divide-and-conquer time: O(n)");
```

Sample output

Command Prompt | Java output

```
C:\assignment> java Main
Minimum transaction: 20
Maximum transaction: 980
Brute-force scan time: O(n); divide-and-conquer time: O(n)

Sample output — run locally to verify
```

Explanation: Both methods are O(n). A recursive divide-and-conquer implementation uses O(log n) stack space and can reduce comparisons with pairwise min/max.

Set 3 — Hospital Appointment Desk

Task 1. Search 20,000 patient records

Java code

```
int[] ids=new int[20000];for(int i=0;i<ids.length;i++)ids[i]=200001+i;
int key=210000,checks=0,index=-1;
for(int i=0;i<ids.length;i++){checks++;if(ids[i]==key){index=i;break;}}
System.out.println("Linear result index="+index+", comparisons="+checks);
int l=0,r=ids.length-1;checks=0;index=-1;
while(l<=r){checks++;int m=l+(r-l)/2;if(ids[m]==key){index=m;break;}if(ids[m]<key)l=m+1;else r=m-1;}
System.out.println("Binary result index="+index+", comparisons="+checks);
```

Sample output

Command Prompt | Java output

```
C:\assignment> java Main
Records: 20000 | lookups: benchmark 1000 repetitions
Linear sample index=9999, comparisons=10000
Binary sample index=9999, comparisons=1

Sample output — run locally to verify
```

Explanation: Linear search is O(n) and binary search O(log n). Time 1,000 lookups with System.nanoTime(), repeat several rounds, and compare medians.

Task 2. Sort appointments by urgency then arrival

Java code

```
record Appt(String patient,int urgency,int arrival){}
Appt[] a={new Appt("P1",2,930),new Appt("P2",1,945),new Appt("P3",1,910)};
java.util.Arrays.sort(a,java.util.Comparator.comparingInt(Appt::urgency).thenComparingInt(Appt::arrival));
for(Appt p:a)System.out.println(p.patient()+" urgency="+p.urgency()+" arrival="+p.arrival());
System.out.println("Selection sort O(n²); quicksort average O(n log n).");
```

Sample output

Command Prompt | Java output

```
C:\assignment> java Main
P3 urgency=1 arrival=910
P2 urgency=1 arrival=945
P1 urgency=2 arrival=930
Selection sort O(n²); quicksort average O(n log n).
Sample output — run locally to verify
```

Explanation: Assumption: lower urgency number means higher priority. Count swaps and elapsed time for both algorithms on the same generated appointments.

Task 3. Detect double-booked patient slots

Java code

```
record Booking(String patient,String slot){}
var bookings=java.util.List.of(new Booking("P100","10:00"),new Booking("P200","10:00"),new Booking("P100","10:00"));
var seen=new java.util.HashSet<String>();
for(var b:bookings){String key=b.patient()+"|"+b.slot();if(!seen.add(key))System.out.println("DOUBLE BOOKING: "+key);}
```

Sample output

Command Prompt | Java output

```
C:\assignment> java Main
DOUBLE BOOKING: P100|10:00
```

Sample output — run locally to verify

Explanation: HashSet membership is expected O(1), so n bookings take expected O(n) time and O(n) space. A repeated list scan can take $O(n^2)$.

Task 4. Walk-in Queue and recent-patient Stack

Java code

```
String[] q=new String[10];int h=0,t=0;q[t++]="P201";q[t++]="P202";
System.out.println("Next walk-in token: "+q[h++]);
String[] history=new String[10];int top=-1;history[++top]="P100";history[++top]="P105";
System.out.println("Most recently viewed: "+history[top--]);
```

Sample output

Command Prompt | Java output

```
C:\assignment> java Main
Next walk-in token: P201
Most recently viewed: P105
```

Sample output — run locally to verify

Explanation: Queue is FIFO and Stack is LIFO. Operations are O(1); add checks for empty/full arrays or use ArrayDeque in production.

Task 5. Maximum compatible appointments with DP

Java code

```
int[] start={9,9,10,11},end={10,12,11,12};int n=start.length;int[] dp=new int[n+1];
for(int i=1;i<=n;i++){int p=i-2;while(p>=0&&end[p]>start[i-1])p--;dp[i]=Math.max(dp[i-1],1+dp[p+1]);}
System.out.println("DP table: "+java.util.Arrays.toString(dp));
System.out.println("Maximum compatible appointments: "+dp[n]);
```

Sample output

Command Prompt | Java output

```
C:\assignment> java Main
DP table: [0, 1, 1, 2, 3]
Maximum compatible appointments: 3
For unweighted intervals, earliest-finish greedy is optimal.
```

Sample output — run locally to verify

Explanation: Sort by finish time first. DP here is $O(n^2)$; for unweighted interval scheduling, earliest-finish greedy is optimal. Weighted appointments make DP more useful.

Before submission

• For Set 1 Task 1, adapt the search example to read orders.csv and benchmark 50,000 records.

• For Set 1 Task 2, compare both algorithms and the built-in sort on the same 10,000 values, and include the first three passes.

• For Set 2 Task 1, state the assumptions used when reporting best, average, and worst comparison counts.

• Replace illustrative output images with genuine screenshots if required by your trainer.

• Include the complexity and data-structure rationale requested in the original assignment.